Chapter 1: What Is AI QA And Why It Is Different

Series: From Manual Tester to AI QA Practitioner: A Structured Six Week Tutorial Series on Testing LLMs and AI Agents
Author: Tanvi Mittal
Series DOI: https://doi.org/10.5281/zenodo.18972360
Chapter: 1 of 6 **Week:** 1 **Estimated Time:** 3-4 hours
License: Creative Commons Attribution 4.0 International (CC BY 4.0)

How to Cite This Chapter

Mittal, T. (2026). *Chapter 1: What Is AI QA And Why It Is Different*. In From Manual Tester to AI QA Practitioner: A Structured Six Week Tutorial Series on Testing LLMs and AI Agents (v1). Zenodo. https://doi.org/10.5281/zenodo.18972360

Learning Outcomes

By the end of this chapter, you will be able to:

- Explain what AI QA is and how it differs from traditional software testing
- Identify the three core properties of AI systems that make them uniquely difficult to test
- Describe the role of an AI QA practitioner on a team that builds or uses AI products
- Recognize at least five real world failure modes that are unique to LLM based systems
- Complete your first structured AI test observation and document it in a professional format

---

1. Where You Are Starting From And Why That Is an Advantage

If you are a manual tester reading this, you already know something important: software breaks in ways its creators did not expect, users behave in ways the product team did not anticipate, and the gap between what a system is supposed to do and what it does is where your professional value lives.

That instinct the habit of asking "but what if?" is exactly what AI QA requires. And it is something no amount of technical training can manufacture from scratch.

What this series will do is give you a new vocabulary, a new set of tools, and a structured way to apply the judgment you already have to a different kind of system. AI systems do not work the way traditional software does. But they fail in ways that are observable, documentable, and testable and that is your domain.

---

2. What Traditional QA Looks Like

Before we define AI QA, it helps to be precise about what traditional software testing involves, because the contrast is the point.

In traditional software testing, you are verifying that a system behaves according to a specification. A button click should trigger a specific function. A form submission with an empty required field should return a specific error message. An API endpoint called with a valid token should return data in a defined format.

The system's behavior is **deterministic** meaning the same input, under the same conditions, produces the same output every time. Your job as a tester is to confirm that the actual output matches the expected output, find cases where it does not, and report them with enough detail for a developer to reproduce and fix the problem.

This model of testing rests on three assumptions:

1. There is a specification a defined description of how the system should behave
2. The correct output for a given input can be known in advance
3. The same test, run twice, should produce the same result

All three of these assumptions break when you are testing AI systems.

---

3. What Changes When the System Is AI

3.1 Non Determinism

An LLM a Large Language Model, which is the technology behind tools like ChatGPT, Claude, and Gemini does not produce the same output every time you give it the same input. It is designed not to. The same prompt submitted twice may produce two responses that are factually consistent but worded differently, structurally different, or in some cases meaningfully different in content.

This is not a bug. It is an intentional property of how these systems are built. But it means that the core assumption of traditional testing same input, same output no longer holds.

As an AI QA practitioner, you shift from asking "does this output match the expected output?" to asking "does this output fall within acceptable behavior?"

That is a more complex question. It requires you to define what "acceptable" means before you test, and to apply judgment when evaluating results judgment that cannot always be automated.

3.2 Emergent Behavior

Traditional software does what it was programmed to do. If a developer did not write code to handle a particular input, the system either throws an error or ignores it. The behavior set is bounded by what was explicitly built.

LLMs exhibit **emergent behavior** they respond to inputs that their developers never specifically anticipated, in ways that were never explicitly programmed. An LLM trained to be a customer support assistant may, under certain conditions, produce medical advice, write code, or adopt a persona its developers did not intend. Not because anyone programmed those behaviors but because they emerge from the model's training.

This means that as a tester, you cannot rely solely on the product specification to know what to test. You need to explore beyond the intended use case and probe for behaviors the developers did not plan for.

3.3 Data Dependency

Traditional software behaves the same regardless of when it was deployed, as long as the code has not changed. An AI system's behavior is shaped by the data it was trained on data that has a cutoff date, that reflects the biases present in the sources it came from, and that cannot be fully inspected by a tester.

This means that AI systems can produce outputs that are outdated (the model's knowledge does not include recent events), biased (the model reflects patterns from its training data that may not be appropriate), or inconsistent across topics (the model may have been trained on much more data about some subjects than others).

These are quality risks that exist at the level of the training data, not at the level of the code and they require a different kind of testing mindset to surface.

---

4. What AI QA Is

AI QA is the practice of systematically evaluating AI based systems to determine whether they behave safely, accurately, consistently, and in alignment with their intended purpose and to identify and document the ways in which they do not.

It sits at the intersection of traditional software quality assurance, data quality assessment, and a newer discipline called AI evaluation (often called "evals" in the industry). It draws on techniques from security testing, behavioral testing, and human factors evaluation.

An AI QA practitioner does not need to be able to build an AI model. What they need to be able to do is:

- Understand what the AI system is supposed to do well enough to test it meaningfully
- Design test inputs (called **prompts** when the input is text) that probe the system's behavior across expected, boundary, and adversarial conditions
- Evaluate outputs against defined criteria and document when those criteria are not met
- Communicate quality risks to product teams, developers, and stakeholders in a clear and actionable way

The role is partly analytical, partly investigative, and partly communicative. These are all skills that experienced manual testers already possess.

---

5. The AI QA Practitioner on a Team

In most organizations that are building or integrating AI products, AI QA does not yet have a formally defined place in the team structure. This is changing rapidly, but it means that in your early career as an AI QA practitioner, you will often need to explain your role to people who have not worked with one before.

Here is the simplest way to frame it:

A software development team has developers who build the system, product managers who define what the system should do, and QA engineers who verify that the system does what it should. When the system is AI powered, the QA function becomes more complex because the outputs are not deterministic, the failure modes are different, and the risks include not just bugs but safety failures, bias, and manipulation. The AI QA practitioner is the person who holds that complexity and builds the processes to manage it.

In practice, this means the AI QA practitioner is involved in:

- **Defining evaluation criteria** before a feature is built what does good output look like?
- **Designing test scenarios** that cover expected use, edge cases, and adversarial use
- **Running test campaigns** against new model versions or prompt changes
- **Documenting behavioral baselines** so that regressions can be detected
- **Escalating safety and quality risks** with evidence that is clear and reproducible

---

6. Five Failure Modes You Will Learn to Test For

The following five failure modes appear repeatedly across every LLM based product category. You will encounter all of them in this series. Each one requires a different testing approach, and each one represents a real quality risk in deployed AI products.

6.1 Hallucination

An LLM produces content that is fluent, confident, and plausible but factually incorrect. It may cite a paper that does not exist, describe a historical event inaccurately, or provide incorrect instructions for a technical process. The model does not know it is wrong. It generates the response because the pattern of words was statistically likely given its training not because it verified the facts.

Testing for hallucination requires you to design inputs where the correct answer is known, evaluate the model's response against that known answer, and document the discrepancy.

6.2 Behavioral Drift

An LLM is given a specific role or set of instructions for example, "you are a customer support agent for a software company, answer only questions about our product." Over the course of a conversation, the model drifts outside that role: it starts giving general advice, adopts a different tone, or responds to topics it was instructed to ignore.

Testing for behavioral drift requires you to design multi turn conversations that gradually push the model toward the boundary of its defined role and observe where and how it breaks.

6.3 Boundary Violations

An LLM is instructed not to do something that produces certain types of content, discuss certain topics, or take certain actions. Under specific conditions, it does it anyway. This may happen because of a cleverly worded input, because of context accumulated over a long conversation, or because the instruction was ambiguous.

Testing for boundary violations is closely related to security testing and is one of the most important skill areas in AI QA.

6.4 Safety Failures

An LLM produces output that could cause harm harmful instructions, dangerous advice, discriminatory content, or content that violates platform policies. Safety failures may emerge from the base model behavior, from insufficient guardrails, or from deliberate adversarial inputs.

Testing for safety failures requires both structured test cases against known risk categories and exploratory testing to surface unexpected failure paths.

6.5 Evaluation Gaps

The team building the AI product does not have a defined way to measure whether the system is working well. There are no evaluation criteria, no scoring rubrics, and no baselines. When the model changes because the underlying model is updated, or because the prompt was modified there is no way to know whether quality improved or degraded.

Identifying evaluation gaps is itself a quality contribution. Part of the AI QA practitioner's role is to build the measurement infrastructure that makes quality visible.

---

7. A Note on Vocabulary

AI QA has its own vocabulary. You will encounter these terms throughout this series. Here are the most important ones to know before you do the exercise:

Term	Plain Language Definition
Prompt	The input you send to an LLM usually a text instruction or question
Completion	The LLM's response to a prompt
System prompt	Instructions given to the LLM by the developer, before the user interacts with it the user often cannot see these

Context window	The total amount of text an LLM can "see" at one time older parts of a long conversation may fall outside this window
Temperature	A setting that controls how varied the model's outputs are higher temperature means more variation
Hallucination	When an LLM produces confident but factually incorrect output
Guardrail	A mechanism that prevents the LLM from producing certain types of output
Grounding	Connecting the LLM's output to verified external information (documents, databases) to reduce hallucination
Eval	Short for evaluation a structured test or scoring process applied to LLM outputs
Token	The unit the LLM processes text in roughly equivalent to three quarters of a word on average

A full glossary for the series is available at `resources/glossary.md` in the series repository.

---

8. Hands On Exercise: The Inconsistency Audit

What You Need

- A free account on any one of the following: ChatGPT (chat.openai.com), Claude (claude.ai), or Gemini (gemini.google.com)
- The exercise template below (copy it into a text file, spreadsheet, or paper notebook)
- 45-60 minutes

What You Are Going to Do

You are going to submit the same prompt to an LLM five times, record every response, and analyze the differences. This exercise builds the foundational AI QA skill of **behavioral observation under repetition** the ability to detect non determinism and characterize it precisely.

Step 1: Choose Your Prompt

Use exactly this prompt, word for word:

> *"A customer has contacted support saying their order has not arrived after 10 days. What should the support agent do? Give me three specific steps."*

This prompt is intentionally straightforward. It has a clear intent and a broadly knowable correct structure. That makes variation easy to see and characterize.

Step 2: Submit and Record

Submit the prompt five times. Between each submission, start a **new conversation** do not continue in the same chat thread. This ensures each response is independent.

Copy each full response into your log. Do not edit or paraphrase the responses at this stage.

Step 3: Analyze the Differences

For each pair of responses, identify differences across these four dimensions:

Dimension	What to Look For
Wording	Same meaning, different words
Structure	Different ordering of steps, different number of sub points
Content	One response includes information the other does not
Factual variation	Responses that contradict each other on a specific point

Step 4: Write Your Defect Note

Find the single most significant difference across your five responses. Write a defect note using this format:

Defect Note Template

Observation ID: CH01 EX01 [your initials] [date]
System Under Test: [Name of LLM and platform you used]
Test Input (Prompt): [Copy the exact prompt]
Observed Behavior: [Describe specifically what varied across responses]
Expected Behavior: [Describe what consistent, acceptable behavior would look like]
Severity Assessment: [Choose one: Low / Medium / High and write one sentence explaining your choice]
Evidence: [Paste the two or more responses that best illustrate the variation]

Step 5: Reflect

Write two to three sentences answering this question:

> *If this LLM were deployed as a real customer support assistant, and your test showed this level of inconsistency, what would you recommend to the product team?*

There is no single correct answer. This reflection builds the habit of translating observations into actionable quality recommendations one of the most important skills in the role.

What to Submit / Save

Save your completed log and defect note. You will reference this exercise when you reach Chapter 6 (capstone), and it is a concrete work sample you can use to demonstrate hands on AI QA experience.

---

9. Key Takeaway

AI systems do not have a single correct output. Your job as an AI QA practitioner is not to find the right answer it is to **define what acceptable behavior looks like**, and then verify whether the system stays within those boundaries consistently.

That shift from verifying correctness to defining and testing acceptability is the foundational conceptual move in AI QA. Everything in the remaining five chapters builds on it.

Chapter Summary

Concept	One Line Summary
Non determinism	Same input, different outputs by design
Emergent behavior	AI systems do things their developers never explicitly programmed
Data dependency	AI quality is shaped by training data, not just code
AI QA definition	Systematic evaluation of AI systems for safety, accuracy, consistency, and alignment
Five failure modes	Hallucination, behavioral drift, boundary violations, safety failures, evaluation gaps

---

What Is Coming in Chapter 2

In Chapter 2, you will go deeper into understanding the system you are testing. You will learn what an LLM actually is at the level of detail a tester needs without the mathematics and you will be introduced to AI agents: systems where an LLM does not just respond to prompts, but takes actions in the world. Understanding that distinction is essential before you can design meaningful test cases for either.

References and Further Reading

The following resources informed the concepts in this chapter and are recommended for learners who want to go deeper:

- OWASP. (2025). *OWASP Top 10 for Large Language Model Applications 2025*. https://owasp.org/wwwproject top 10 for large language model applications/
- Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. https://doi.org/10.18653/v1/2020.acl main.442
- Garg, S., Perot, V., Limtiaco, N., Taly, A., Chi, E. H., & Beutel, A. (2019). Counterfactual fairness in text classification through robustness. *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*.
- Perez, F., & Ribeiro, I. (2022). *Ignore previous prompt: Attack techniques for language models*. https://arxiv.org/abs/2211.09527
- Anthropic. (2023). *Claude's model specification*. https://anthropic.com/model card

---

This chapter is part of the series: From Manual Tester to AI QA Practitioner (Zenodo DOI: https://doi.org/10.5281/zenodo.18972360). Published under CC BY 4.0. Please cite using the citation block at the top of this document.